

Fire Assignment 12

by Lorraine Gaudio

Lesson generated on November 14, 2025



BOISE STATE UNIVERSITY

Contents

1	☒ Point-Boost Activity	2
1.1	Assignment Directions	2
1.2	Dataset	3
2	Assignment A	5
2.1	Task 1	5
2.2	Task 2	6
2.3	Task 3	6
2.4	Task 4	7
2.5	Task 5	8
2.6	Assignment B	9
2.7	Objective	10
2.8	Lognormal Random Numbers	10
2.9	Task 1	11
2.10	Task 2	11
2.11	Task 3	11
2.12	Task 4	12
2.13	Task 5	12
2.14	Task 6	13
2.15	Task 7	13
2.16	Task 8	14
2.17	Task 9	15
2.18	Task 10	15
2.19	Reflections	16
3	Save and Upload	17
4	Today you practiced:	18

1. ☒ Point-Boost Activity

☐ How far could a new wildfire run in 8 hours? This is an optional two-part point-boost activity. You can work in groups but each person submits their own work independently. Please record group members within your document.

1.1 Assignment Directions

Read the background material below, then complete the tasks that follow. Replace each ____ placeholder (and any TODO comments) with working code or a short written answer. Run each section; be sure the requested objects appear in the Environment.

1. Technical Skill Points

You could earn *up to* **10** points in Technical skill. This assignment will require code covered in modules 1 through 13. You will load a CSV file, manipulate the data frame with dplyr verbs, simulate random numbers and visualize results. Additionally, you will be asked to use functions that were not covered in any modules. You will need to look up the function documentation and figure out how to use them on your own. The submission must include an HTML report and RData of the environment.

2. Learning Skill Points

You could earn up to **10** points Learning skill on this optional Point Boost assignment!

- Use the template ☐ Goal → ☐ Code → ☐ Checks → ☐ Explain.
- ☐ Cite resources (including referencing lessons, peers, instructor guidance, internet websites, AI summaries, the help file in R) in memos. State specifically what or who helped and in what ways. Always compose your own code (no copy and paste).
- ☐ Reflect on learning process, connects ideas, discusses challenges, and describes improvement strategies.
- ☐ Format your document with headings, code chunks, and narrative text. Use proper grammar, spelling, and punctuation. Include a document title, author name, and ☐ date in the heading of your report.
- ☐ Code trials show how you explored ideas, asked questions independently of the assignment. Include code comments to explain your thinking.
- ☐ Growth mindset means you demonstrate effort, persistence, and a positive attitude toward learning new skills. You show courage.

3. Signature Assignment Option

You may choose to direct these points to your Signature Assignment requirement. If you do, you **must** complete all tasks and submit a polished HTML document with clear code, thorough comments, and well-written explanations.

When finished, ☐ Knit or Render your report to HTML, and upload with your RData to Canvas by or before December 12th.

1.2 Dataset

Wildfire managers need tools that show where fires are more likely and how intense they could become. The U.S. Forest Service's Wildfire Hazard Potential (WHP) is one such tool: a nationwide raster layer where each grid cell stores a unitless index of relative wildfire hazard (higher value = higher hazard). For this activity we use an Idaho-only WHP file prepared by Dr. Matt Williamson [SPASES Lab Page](#).

Figure 1 shows the Idaho WHP raster created in R with the packages `terra` (raster operations), `sf` (vector data), `tidyverse` (data wrangling), `tidyterra` (ggplot helpers for rasters), and `tigris` (state/county boundaries). The GeoTIFF uses an Albers Equal Area projection (units: meters) with 240×240 m cells. Within Idaho, valid cells cover about 216,315 km²; cells outside the state within the file's bounding box are NoData. WHP values in this subset range from 0 to 64,185.656; the median is about 397, and the upper tail is steep (e.g., 95th percentile $\approx 7,753$, 99th $\approx 22,429$).

Formal dataset citation

Dillon, Gregory K.; Gilbertson-Day, Julie W. 2020. Wildfire Hazard Potential for the United States (270-m), version 2020. 3rd Edition. Forest Service Research Data Archive. <https://doi.org/10.2737/RDS-2015-0047-3>.

Idaho subset (`whp_id.tif`) prepared and provided by Matt Williamson, SPASES Lab, Boise State University.

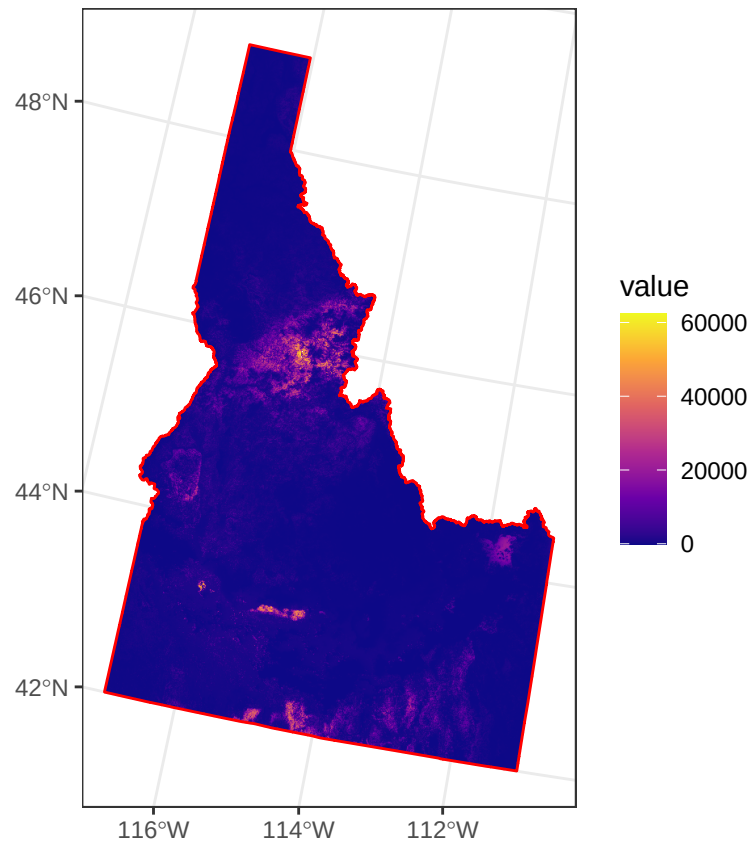


Figure 1.1: Figure 1. Idaho Wildfire Hazard Potential (WHP), continuous index (unitless) from the USFS WHP v2020 raster, clipped to the Idaho boundary. Colors use a viridis gradient (dark purple = lower WHP; yellow = higher WHP); values span approximately 0–64,186. Raster resolution is 240 m in an Albers Equal Area projection; NoData cells are transparent. Red outline shows the TIGER/Line state boundary reprojected to the raster CRS. Idaho subset prepared by Matt Williamson (SPASES Lab, Boise State University); original dataset: Dillon & Gilbertson-Day (2020), USFS Research Data Archive (DOI: 10.2737/RDS-2015-0047-3).

2. Assignment A

Explore and wrangling of the raw WHP sample data.

2.1 Task 1

□ Goal: Read the `whp_raw_sample.csv` using `readr`, storing it in your environment as `county_raw`.

□ Code

```
library(readr)
county_raw <- readr::read_csv("whp_raw_sample.csv")
```

□ *Verify Ideas*

```
dplyr::glimpse(county_raw)
colnames(county_raw)
summary(county_raw$whp)
nrow(county_raw)
```

□ Explain: `county_raw` is a 150,000 row tibble of raw WHP cell values with county and per-cell area.

whp – The Wildfire Hazard Potential value for a single sampled raster cell. This is a unitless, continuous index (higher = greater relative hazard). Values come directly from the WHP GeoTIFF.

county – The county name (e.g., “Idaho”, “Twin Falls”) obtained by spatially joining each sampled cell to the TIGER/Line county boundary. Use for readable reports.

county_fips – The county’s 3-digit FIPS code from TIGER/Line. *Option: To create the standard 5-digit FIPS for Idaho, prepend the state code 16: e.g., 049 → 16049. Need to use the 5-digit code when joining to external datasets (ACS, MTBS summaries, etc.).*

cell_area_km2 – The area (in square kilometers) represented by that raster cell, computed from the raster’s equal-area projection. With $240\text{ m} \times 240\text{ m}$ cells this is $\sim 0.0576\text{ km}^2$; small numeric differences reflect rounding. Summing this column by any grouping yields true area totals, and dividing by the statewide sum yields area shares.

□ Task 2 through 5 Goal: Compute quantile cut-points, label categories, and estimate probability (statewide area share) from this.

2.2 Task 2

□ Goal: Use `dplyr::summarize()` to report the following about `county_raw`:

- Number of rows
- Number of missing values in `whp`
- Minimum value of `cell_area_km2`
- Maximum value of `cell_area_km2`

```
# □ Code: for dplyr::summarize
county_raw %>%
  summarize(
    n_rows = n(),
    n_na_whp = sum(is.na(whp)),
    min_area = min(cell_area_km2),
    max_area = max(cell_area_km2)
  )
```

□ Explain:

2.3 Task 3

Compute WHP quantile edges (category cutpoints)

□ Goal: Use `quantile()` function from base R's `stats` package to compute the quantiles of the `whp` column in `county_raw`. Store the result as `e` for “edges”.

Create the seven category edges using the quantiles (0, .25, .50, .75, .90, .95, .99, 1.00).

□ The SYNTAX: `quantile(x, probs, na.rm)`

```
# □ Code: Base R stats::quantile
e <- stats::quantile(
  county_raw$whp,
  probs = c(0, .25, .50, .75, .90, .95, .99, 1),
  na.rm = TRUE
)
```

```
# □ Verify
e
```

□ Explain: The `e` object now contains the cut-points for categorizing WHP into quantile-based bins.

- `e[1]` = minimum WHP (0th percentile)
- `e[2]` = 25th percentile

- `e[3]` = 50th percentile (median)
- `e[4]` = 75th percentile
- `e[5]` = 90th percentile
- `e[6]` = 95th percentile
- `e[7]` = 99th percentile
- `e[8]` = maximum WHP (100th percentile)

2.4 Task 4

Add a Category label

□ Goal: Use `dplyr::mutate()` and `dplyr::case_when()` to turn the numeric whp values in `county_raw` into an ordered categorical variable called `Category`, using the quantile edges `e` you computed in Task 3. Store the resultant data set as `county_cats`.

1. Start from `county_raw`.
2. Use `mutate()` to add a new variable `Category`.
3. Use `case_when()` to assign a label based on which interval of `e` each whp value falls into.
4. Convert `Category` to an ordered factor so the levels follow the intended risk order.

Each row will be assigned exactly one of these labels based on its whp value:

- If a row's whp value is *at least* the minimum WHP AND *less than* the 25th percentile, label it "Very Low".
- If whp is *at least* the 25th percentile AND *less than* the 50th percentile (median), label it "Low".
- If whp is *at least* the 50th percentile AND *less than* the 75th percentile, label it "Moderate".
- If whp is *at least* the 75th percentile AND *less than* the 90th percentile, label it "High".
- If whp is *at least* the 90th percentile AND *less than* the 95th percentile, label it "Very High".
- If whp is *at least* the 95th percentile AND *less than* the 99th percentile, label it "Extreme".
- If whp is *at least* the 99th percentile AND *less than or equal to* the maximum WHP, label it "Ultra (Top 1%)".

If a value of whp does not fall in any of these ranges (e.g., NA), `Category` should be NA.

Hint: All intervals are closed on the left ($\geq$) and open on the right ($<$), except the last one, which is closed on both sides ($\geq$ and $\leq$).


```
# □ Code: dplyr::mutate + dplyr::case_when
county_cats <- county_raw %>%
  mutate(
    Category = case_when(
      whp >= e[1] & whp < e[2] ~ "Very Low",
      whp >= e[2] & whp < e[3] ~ "Low",
      whp >= e[3] & whp < e[4] ~ "Moderate",
      whp >= e[4] & whp < e[5] ~ "High",
      whp >= e[5] & whp < e[6] ~ "Very High",
      whp >= e[6] & whp < e[7] ~ "Extreme",
      whp >= e[7] & whp <= e[8] ~ "Ultra (Top 1%)",
      TRUE ~ NA_character_
    ),
    Category = factor(
      Category,
      levels = c("Very Low", "Low", "Moderate", "High", "Very High", "Extreme", "Ultra (Top 1%)",
      ordered = TRUE
    )
  )
)
```

```
# □ Verify
county_cats %>% count(Category)
```

□ Explain: I used `case_when` to assign labels by WHP range. Because the CRS is equal-area and cell size is uniform, counts approximate area. The `cell_area_km2` column lets you compute true area easily.

2.5 Task 5

□ Goal: Use `dplyr::group_by()` and `dplyr::summarize()` to aggregate total area by Category, then compute Probability as each category's share of the total statewide area. Store the result as `prob_tbl`.

1. Start from `county_cats` (the data set you created in Task 4, which already has `Category` and `cell_area_km2`).
2. Use `group_by()` so that all rows in the same `Category` are treated as a single group.
3. Use `summarize()` to compute, for each `Category`:
 - `area_km2`: the total area in square kilometers, as the sum of `cell_area_km2` across all cells in that category.
 - `n_cells`: the number of grid cells in that category using `n()`.
 - Use `.groups = "drop"` so that the result is no longer grouped after summarizing.
4. Use `mutate()` to create a new variable called `Probability` that is the `area_km2` divided by `sum(area_km2)`.

This makes Probability the fraction of statewide area that falls in each Category. The probabilities across all categories should sum to 1 (up to rounding error).

5. Use `select()` to keep only the columns, in this order: Category, Probability, area_km2, n_cells.

```
# □ Code: dplyr::group_by + dplyr::summarize + dplyr::mutate
prob_tbl <- county_cats %>%
  group_by(Category) %>%
  summarize(
    area_km2 = sum(cell_area_km2),
    n_cells = n(),
    .groups = "drop"
  ) %>%
  mutate(
    Probability = area_km2 / sum(area_km2)
  ) %>%
  select(Category, Probability, area_km2, n_cells)
```

```
# □ Verify
prob_tbl
sum(prob_tbl$Probability)
```

- Explain: The `prob_tbl` tibble now summarizes total area and cell counts by WHP category, along with the statewide area share (Probability) for each category.

2.6 Assignment B

The table below summarizes the quantile-based wildfire hazard categories derived from Idaho's continuous USFS WHP raster and the distance distributions used for an 8-hour run. The **Category** column lists seven bins (Very Low to Ultra) defined from the raster's cumulative distribution; the WHP index itself is unitless. The **Probability** column is the statewide area share for each bin—expressed as a percent of valid raster area (NA excluded)—and is interpreted as the chance that a random ignition falls in that category.

The table also defines a simple fire-spread scenario using plausible values for an 8-hour straight-line distance by hazard category. These distances are not estimated from the WHP raster; they are modeling assumptions chosen for this assignment. The **Normal mean (km)** and **Normal SD (km)** columns give the mean and standard deviation, in kilometers, for a zero-truncated normal distance model representing straight-line spread over 8 hours. The **Mean log** and **SD log** columns give the parameters μ and σ of a lognormal distance model in which

$$\log(\text{distance in km}) \sim \mathcal{N}(\mu, \sigma^2).$$

Labels “Extreme” and “Ultra” denote upper-tail bins (approximately the 95–99% and 99–100% quantiles, respectively) and do not correspond to any official USFS WHP class names.

Category	Probability	Mean (km)	SD (km)	Mean log	SD log
Very Low	25	2.0	1.0	0.582	0.472
Low	25	3.0	1.0	1.046	0.325
Moderate	25	4.5	1.5	1.451	0.325
High	15	7.0	2.0	1.907	0.280
Very High	5	12.0	3.0	2.455	0.246
Extreme	4	18.0	4.0	2.866	0.220
Ultra (Top 1%)	1	28.0	6.0	3.310	0.212

2.7 Objective

Objective. Estimate short-term spread risk using simulation. Model each ignition’s 8-hour straight-line distance with a lognormal distribution. Do this two ways:

Statewide mixture: sample a WHP category using the Probability column in Table 1, then draw a distance from that category’s Mean log and SD log. An ignition critical if it distance ≥ 20 km

You will simulate 2,000 seasons, each with 450 ignitions. For each season, count how many ignitions are ≥ 10 km (major runs) and how many are critical under the rule. Summarize both expected counts and the probability the season exceeds a planning threshold (e.g., “> 50 major runs” and “> 10 critical runs”). Distance is used here as a proxy for severity; in practice agencies track area. Category probabilities approximate where ignitions occur and are a teaching simplification.

2.8 Lognormal Random Numbers

The lognormal model is used because most fires will move a modest distance, but a small fraction travel much farther. A lognormal model enforces non-negativite distance values and captures that right tail.

Use `rlnorm()` to generate lognormal random numbers.

```
?rlnorm # Check the help file for rlnorm()
```

□ The SYNTAX: `rlnorm(n, meanlog = 0, sdlog = 1)`

- `n` is the number of random numbers to generate. In this scenario, `n` is the number if ignitions to simulated per Monte Carlo run.
- `meanlog` is the mean of the logarithm of the distribution with default set to 0.
- `sdlog` is the standard deviation of the logarithm of the distribution with default set to 1.

Note: You should always use the log-scale parameters of mean and SD in `rlnorm()`.

2.9 Task 1

□ Goal: Start by setting a seed. Generate a large vector of simulated 8-hour distances for the **High** category using `rlnorm()`. Look up the **High** row's **Mean log** and **SD log** in **Table 1**, and store the result as `High_Lognorm`. Set the number of ignitions per season to 450.

```
set.seed(12345)
# □ Test
High_Lognorm <- rlnorm(n = 450, meanlog = 1.907, sdlog = 0.280)
```

2.10 Task 2

□ / □ Checks and Comment

Summarize what you just simulated. Report the median, mean, and 90th percentile of `High_Lognorm`, and comment in one sentence on the right-skew you expect for a lognormal.

```
# □ Verify
median(High_Lognorm)
```

```
## [1] 7.035328
```

```
mean(High_Lognorm)
```

```
## [1] 7.224904
```

```
quantile(High_Lognorm, 0.9)
```

```
## 90%
## 10.03975
```

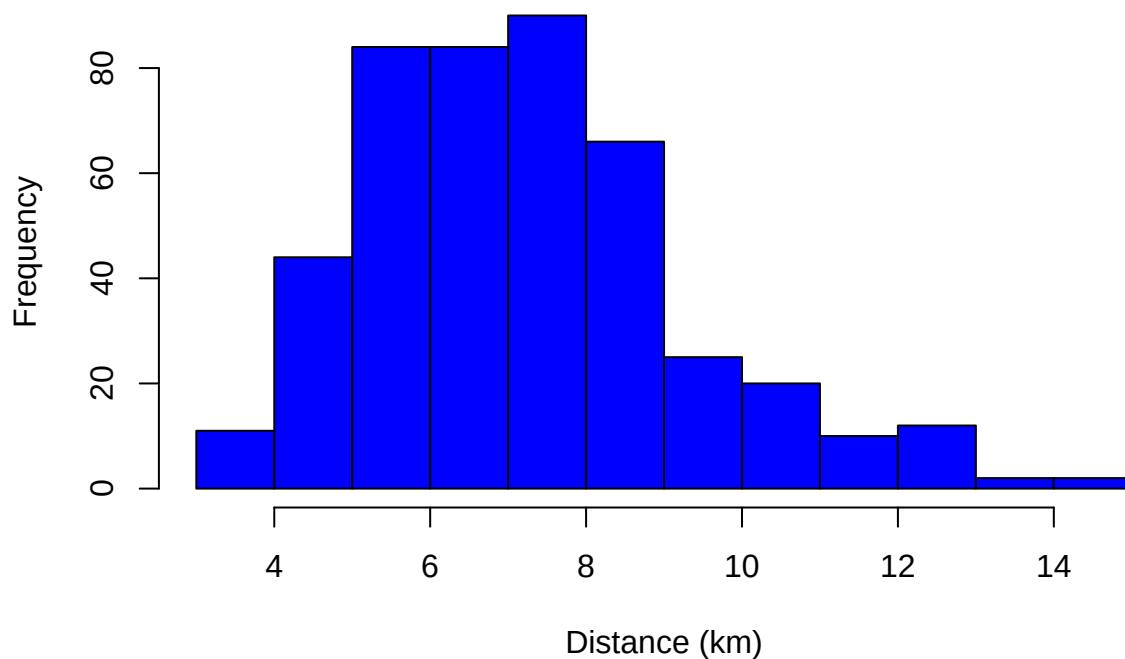
2.11 Task 3

□ / □ Checks and Comment

Make a histogram of `High_Lognorm`. In one sentence, note how the shape matches your summary in Task 2.

```
# □ Verify
hist(High_Lognorm,
     col = "blue",
     main = "Simulated 8-Hour Distances for High WHP (Log
Normal Model)",
     xlab = "Distance (km)")
```

Simulated 8-Hour Distances for High WHP (Log Normal Model)



2.12 Task 4

□ Goal: Copy the code that created High_Lognorm and nest it inside of the mean() function. Set the rlnorm() call to >= 10. This one line will compute the proportion of a single sample that reaches or exceeds a threshold of 10 km spread in 8-hours. Store this result as p_high_one

```
# □ Test
p_high_one <- mean(rlnorm(n = 450, meanlog = 1.907, sdlog = 0.280) >= 10) # one season, 450 ignitions
```

2.13 Task 5

□ / □ Checks and Comment

Comment about meaning of p_high_one. What does this value represent in the context of wildfire spread risk for ignitions in the **High** WHP category?

```
# □ Verify
p_high_one
```

```
## [1] 0.07333333
```

2.14 Task 6

□ Goal: Turn that single-season calculation into many seasons. Use `replicate()` to repeat the same inside expression for 2,000 seasons. Store the resulting vector of proportions as `p_high_many`, then report its mean and make a small histogram of those season-level proportions.

```
# □ Test
p_high_many <- replicate(
  n = 2000,                                # 2,000 seasons
  expr = mean(rlnorm(n = 450, meanlog = 1.907, sdlog = 0.280) >= 10) # 450 wildfires, 10 km threshold
)
```

```
# □ Verify
mean(p_high_many)
```

```
## [1] 0.07926556
```

```
mean(p_high_many >= 1)
```

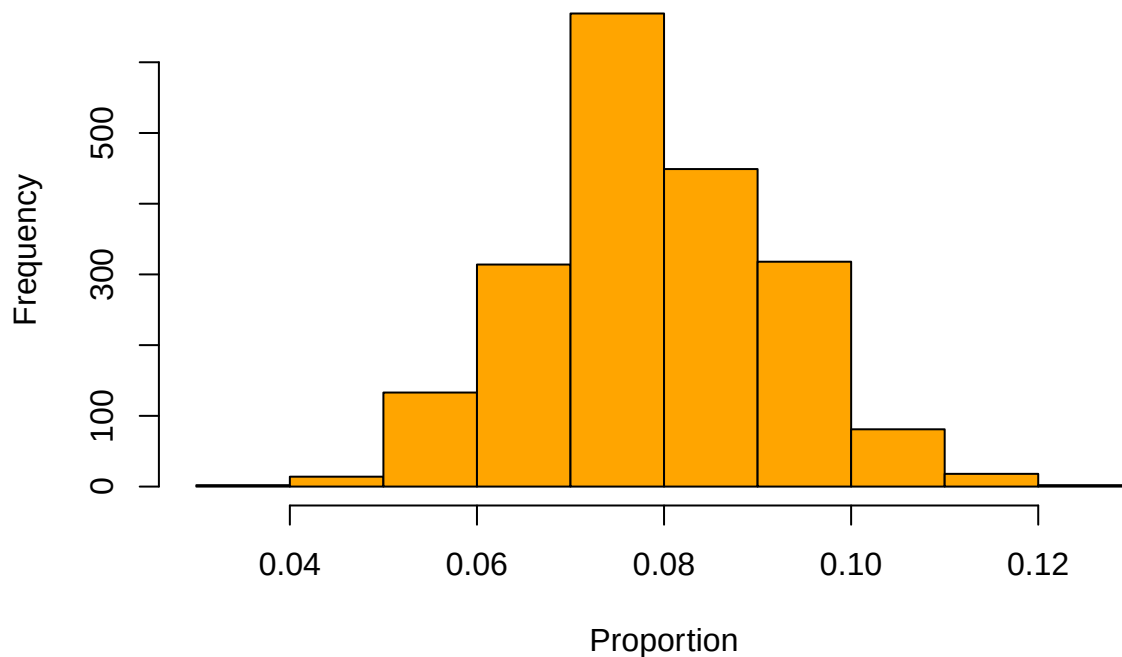
```
## [1] 0
```

2.15 Task 7

□ / □ Checks and Comment

```
# □ Verify
hist(p_high_many,
  col = "orange",
  main = "Season-level proportions  $\geq k$  (High)",
  xlab = "Proportion")
```

Season-level proportions = k (High)



2.16 Task 8

□ Goal: Create four parallel vectors from Table 1: `cats`, `prob` (as proportions that sum to 1), `meanlog`, and `sdlog` in matching order. Build the inside expression for a single season of $N <- 450$ ignitions: first sample a category index for each ignition using `sample()` with `prob`, then draw distances with `rlnorm()` using the matched `meanlog` and `sdlog`. Return a count of ignitions $\geq k$. Run it once to confirm it works.

```
# □ Test
cats <- c("Very Low", "Low", "Moderate", "High", "Very High", "Extreme", "Ultra")
prob <- c(0.25, 0.25, 0.25, 0.15, 0.05, 0.04, 0.01) # proportions sum to 1
meanlog <- c(0.582, 1.046, 1.451, 1.907, 2.455, 2.866, 3.310)
sdlog <- c(0.472, 0.325, 0.325, 0.280, 0.246, 0.220, 0.212)
N <- 450 # ignitions per season
one_mixture_count <- {
  idx <- sample.int(length(cats), size = N, replace = TRUE, prob = prob)
  d <- rlnorm(N, meanlog = meanlog[idx], sdlog = sdlog[idx])
  sum(d >= 10)
}
one_mixture_count
```

```
## [1] 34
```

2.17 Task 9

□ Goal: Wrap the mixture inside code from Task 7 in `replicate()` for S seasons. Save the season counts as `counts_mix`. Report the expected number of $\geq k$ runs and the probability that a season has more than 10 such runs (change the threshold if instructed).

```
# □ Test
counts_mix <- replicate(
  n = 2000,                                # 2,000 seasons
  expr = {
    idx <- sample.int(length(cats), size = N, replace = TRUE, prob = prob)
    d <- rlnorm(N, meanlog = meanlog[idx], sdlog = sdlog[idx])
    sum(d >= 10)
  }
)
```

2.18 Task 10

□ / □ Checks and Comment

```
# □ Verify
mean(counts_mix)      # expected number of  $\geq k$  runs
```

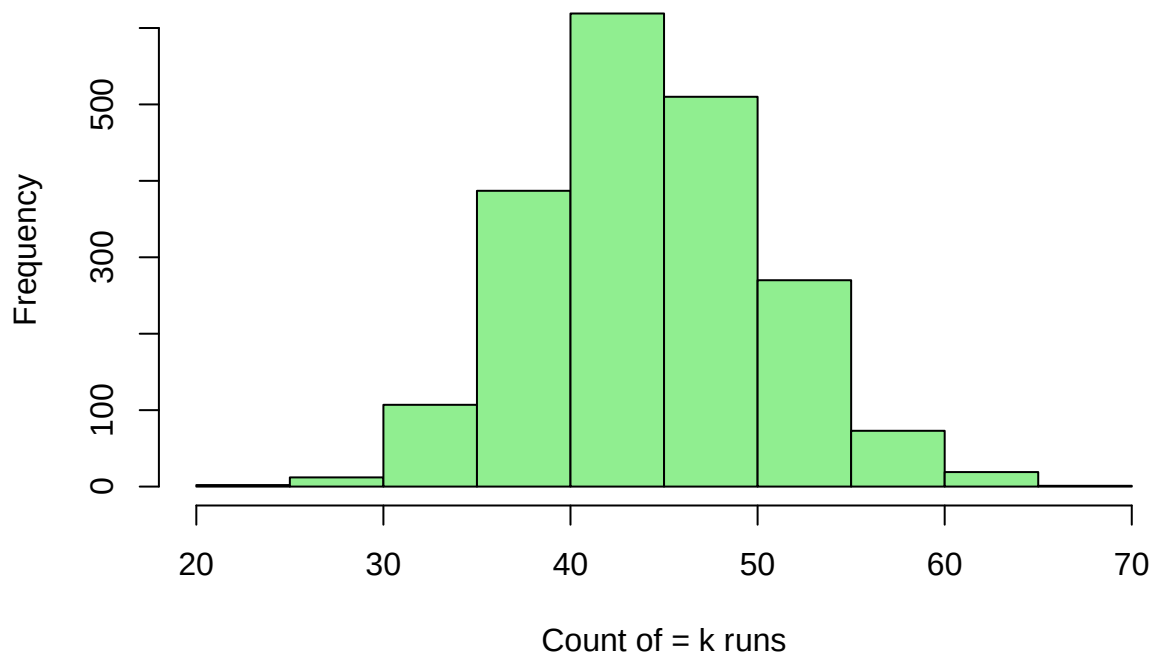
```
## [1] 44.7605
```

```
mean(counts_mix > 10) # probability of > 10 such runs
```

```
## [1] 1
```

```
# □ Verify
hist(counts_mix,
  col = "lightgreen",
  main = "Season-level counts of  $\geq k$  runs (Mixture Model)",
  xlab = "Count of  $\geq k$  runs")
```


Season-level counts of = k runs (Mixture Model)



2.19 Reflections

□ Comment: In 1-2 sentences, reflect on what you learned from this assignment about using simulation to estimate wildfire spread risk. Mention any challenges you faced and how you overcame them.

3. Save and Upload

1. You will be submitting **both** the *Your Pick* Document and the workspace file. The workspace file saves all the objects in your environment that you created in this lesson. You can save the workspace by running the following command in a code chunk of the Quarto Document document:

```
save.image("Assignment12_Workspace.RData")
```

Or you can click the “Save Workspace” button in the Environment pane.

□ **Always save the R documents before closing.**

2. Find the assignment in this week’s module in Canvas and upload **both** the RMD and the workspace file.

4. Today you practiced:

- Used `sample()` for discrete simulations with or without replacement.
- Visualized uniform and aggregated dice results with histograms.
- Generated normal random numbers with `rnorm()` and plotted the shape.
- Repeated an experiment easily with `replicate()`.
- Compared two simulated distributions to spot bias.